

객체탐지

객체탐지 (Object Detection)

- 한 이미지에서 객체와 그 경계 상자(bounding box)를 탐지
- 객체 탐지 알고리즘은 일반적으로 이미지를 입력으로 받고, 경계 상자와 객체 클래스를 출력

Applications

- 자율 주행 자동차에서 다른 자동차와 보행자를 찾을 때
- 의료 분야에서 방사선 사진을 종양이나 위험한 조직을 찾을 때
- 제조업에서 조립 로봇이 제품을 조립하거나 수리할 때
- 보안 산업에서 위협을 탐지하거나 사람 수를 셀 때

객체탐지 (Object Detection)

Bounding Box

- 이미지에서 하나의 객체 전체를 포함하는 가장 작은 직사각형

Bounding Box

x, y, width, height

Class

Car



What is the difference between

Object Classification

Object Detection

Object Segmentation

객체탐지 (Object Detection)

Object Classification



Model



Cat

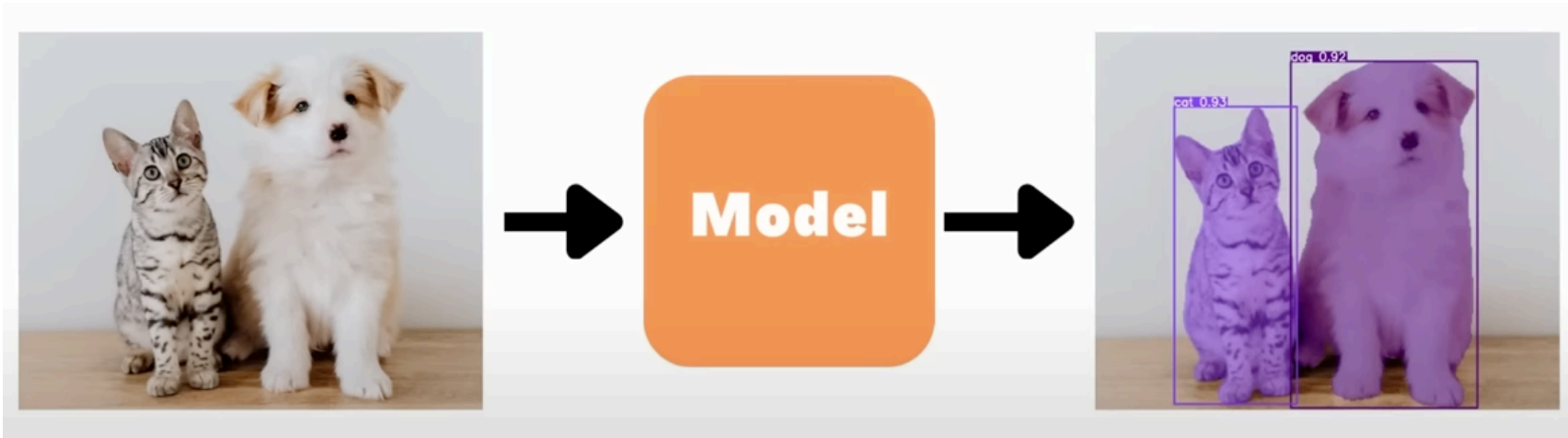
객체탐지 (Object Detection)

Object Detection



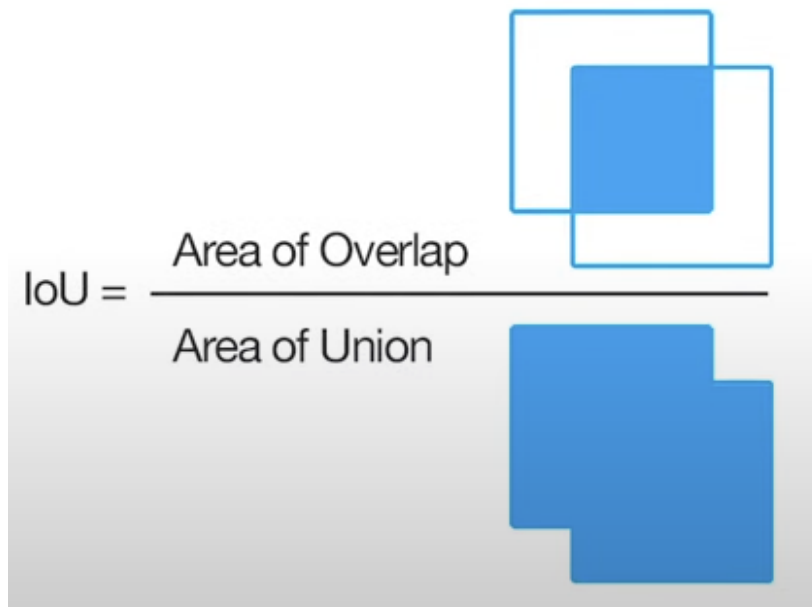
객체탐지 (Object Detection)

Object Segmentation



IOU (Intersection Over Union)

- 실측값(Ground Truth)과 모델이 예측한 값이 얼마나 겹치는지를 나타내는 지표



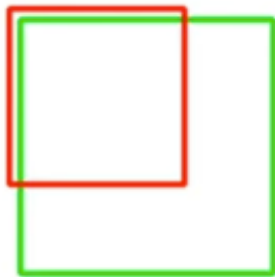
객체탐지 (Object Detection)

IOU (Intersection Over Union)

- 실측값(Ground Truth)과 모델이 예측한 값이 얼마나 겹치는지를 나타내는 지표

IOU가 높을수록 잘 예측한 모델

IoU: 0.4034



Poor

IoU: 0.7330



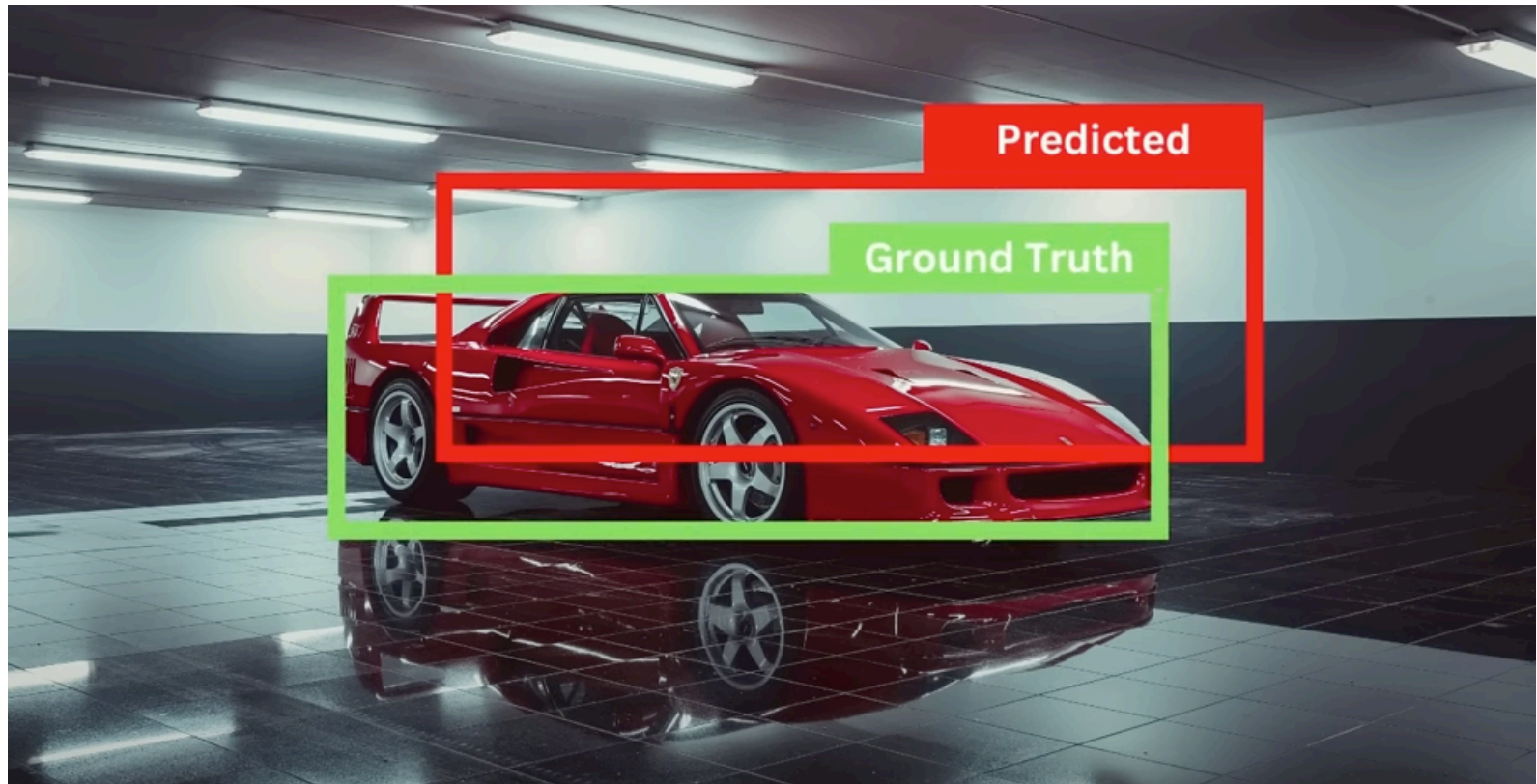
Good

IoU: 0.9264



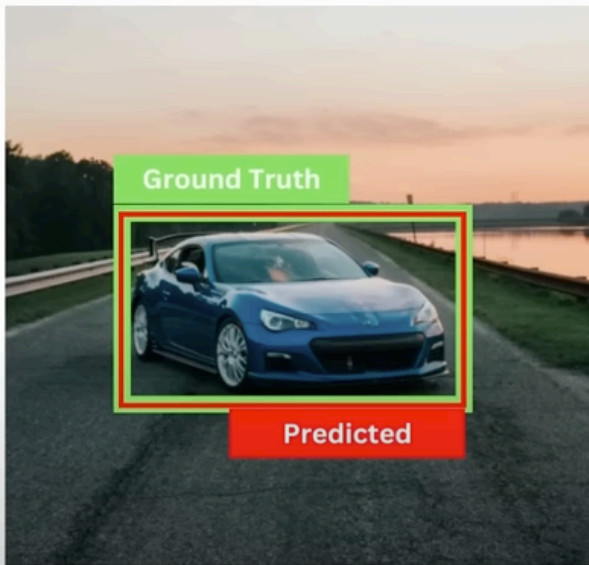
Excellent

객체탐지 (Object Detection)

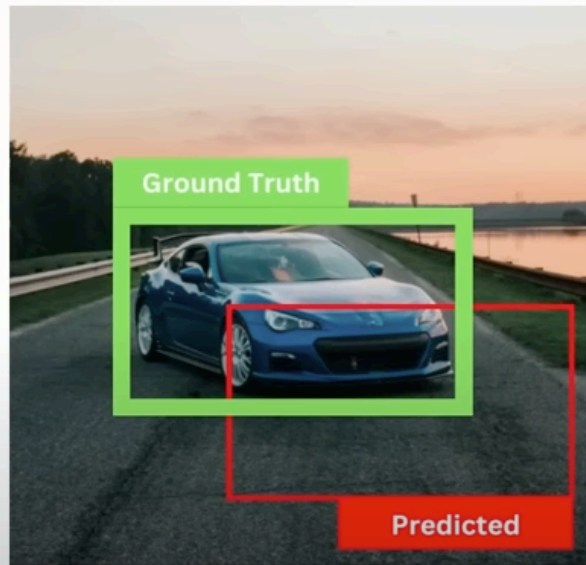


객체탐지 (Object Detection)

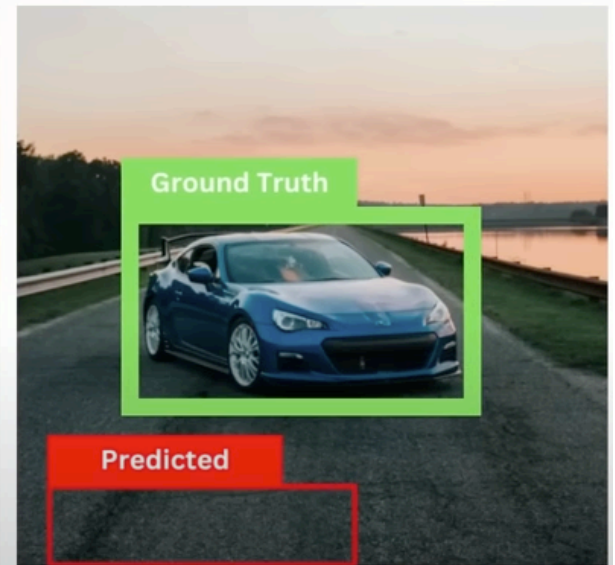
IoU = 1



IoU = 0.4



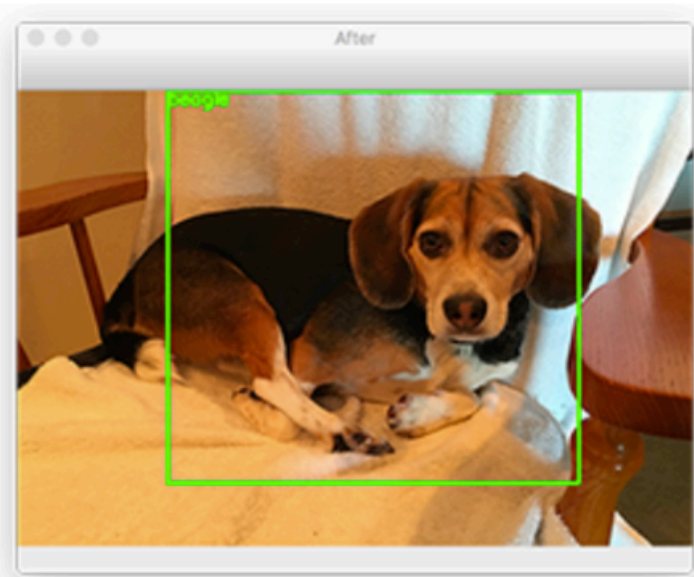
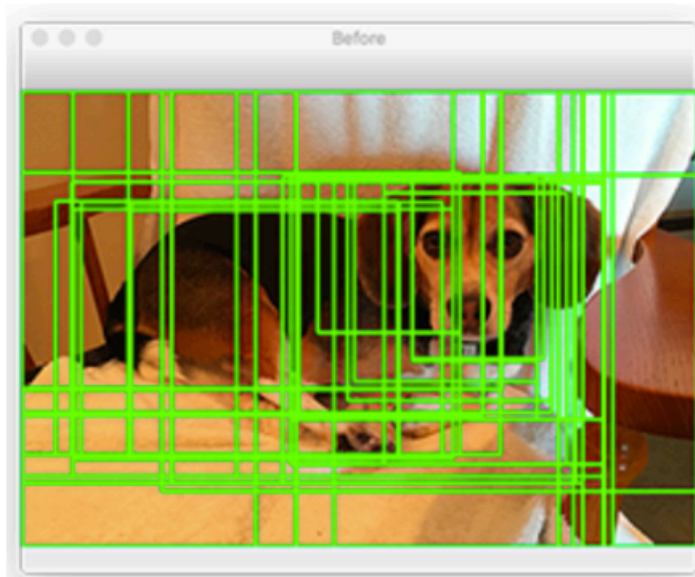
IoU = 0



객체탐지 (Object Detection)

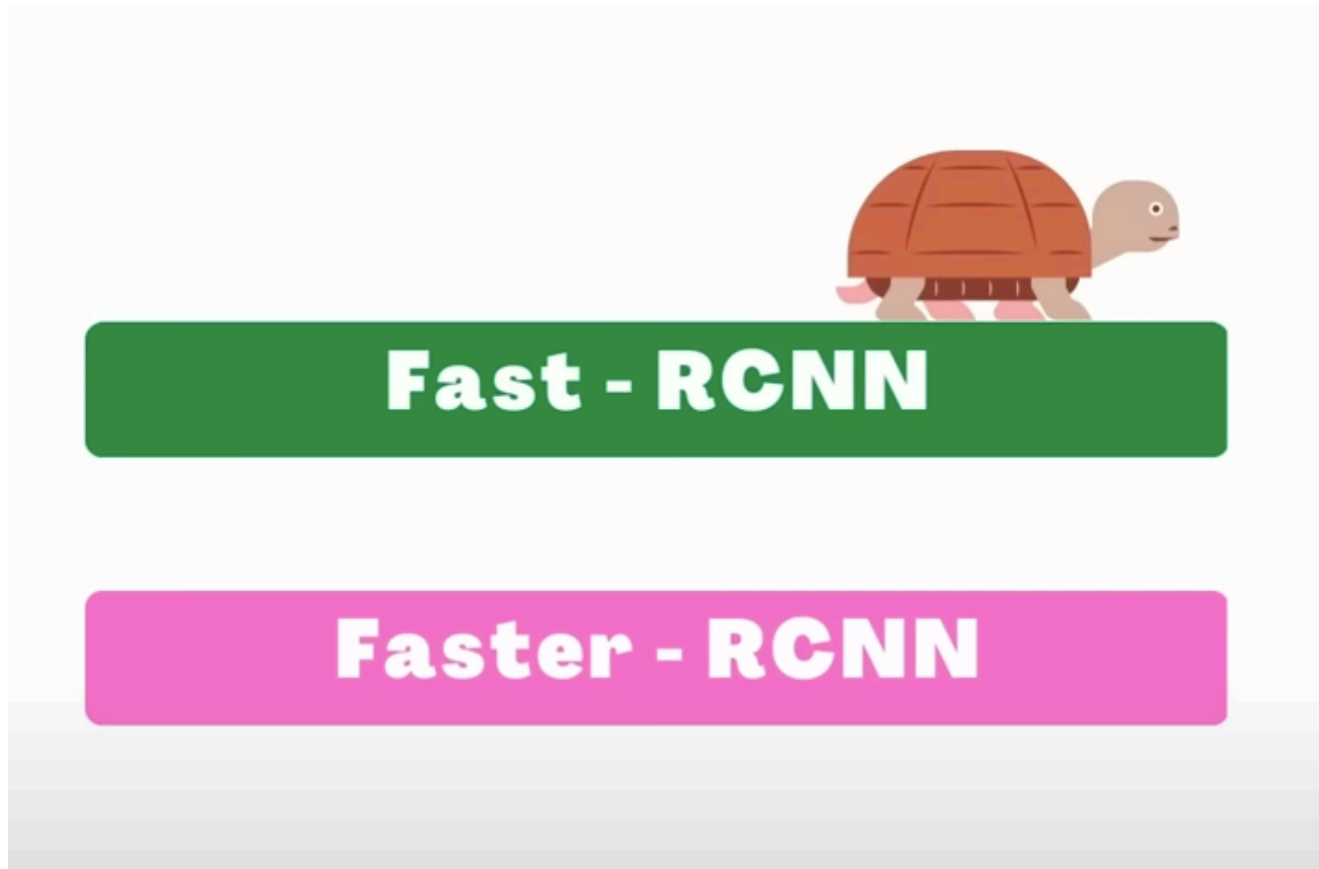
NMS (Non-Maximum Suppression, 비최대값 억제)

- 확률이 가장 높은 상자와 겹치는 상자들을 제거하는 과정
- 최대값을 갖지 않는 상자들을 제거
- 과정
 1. 확률 기준으로 모든 상자를 정렬하고 먼저 가장 확률이 높은 상자를 취함
 2. 각 상자에 대해 다른 모든 상자와의 IOU를 계산
 3. 특정 임계값을 넘는 상자는 제거



객체탐지 (Object Detection)

객체 탐지(Object Detection)의 역사



객체탐지 (Object Detection)

객체 탐지(Object Detection)의 역사

2015

YOLO

객체탐지 (Object Detection)



객체탐지 (Object Detection)

Mask R-CNN (2018)

- Mask R-CNN

YOLO (2018)

- YOLOv3 : An Incremental Improvement
- YOLO는 v1, v2, v3의 순서로 발전하였는데, v1은 정확도가 너무 낮은 문제가 있었고, 이 문제는 v2까지 이어짐
- 엔지니어링적으로 보완한 v3는 v2보다 살짝 속도는 떨어지더라도 정확도를 대폭 높인 모델
- RetinaNet과 마찬가지로 FPN을 도입해 정확도를 높임
- RetinaNet에 비하면 정확도는 4mAP정도 떨어지지만, 속도는 더 빠르다는 장점

객체탐지 (Object Detection)

YOLOv4 (2020)

- YOLOv4 : Optimal Speed and Accuracy of Object Detection
- YOLOv3 에 비해 AP, FPS가 각각 10%, 12% 증가
- YOLOv3와 다른 개발자인 AlexeyBochkovskiy가 발표
- v3에서 다양한 딥러닝 기법 (WRC, CSP ...) 등을 사용해 성능을 향상시킴
- CSPNet 기반의 backbone (CSPDarkNet53)을 설계하여 사용

YOLOv5 (2020)

- YOLOv4에 비해 낮은 용량과 빠른 속도 (성능은 비슷)
- YOLOv4와 같은 CSPNet 기반의 backbone을 설계하여 사용
- YOLOv3를 Pytorch로 implementation 한 GlennJocher가 발표
- Darknet이 아닌 Pytorch 구현이기 때문에, 이전 버전들과 다르다고 할 수 있음

이후

- 수 많은 YOLO 버전들이 탄생, Object Detection 분야의 논문들이 계속해서 나오고 있음

객체탐지 (Object Detection)

YOLO (You Only Look Once)

- 가장 빠른 객체 검출 알고리즘 중 하나
- 256 x 256 사이즈의 이미지
- 파이썬, 텐서플로 기반 프레임워크가 아닌 C++로 구현된 코드 기준 GPU 사용 시, 초당 170 프레임 (170 FPS, frames per second)
- 작은 크기의 물체를 탐지하는데는 어려움

객체탐지 (Object Detection)

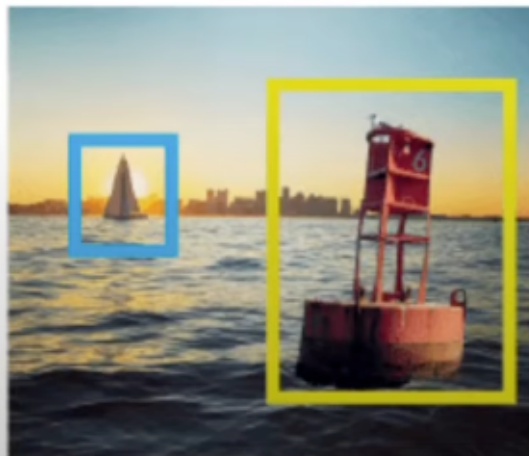
YOLO 아키텍처

- 앵커박스 (Anchor Box)

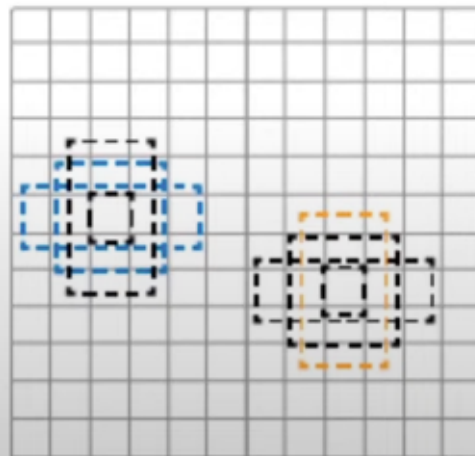
YOLOv2에서 도입

사전 정의된 상자 (prior box)

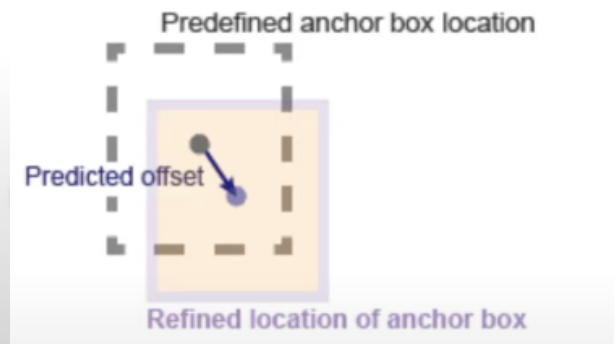
객체에 가장 근접한 앵커 박스를 맞추고 신경망을 사용해 앵커 박스의 크기를 조정하는 과정



Ground truth image and bounding boxes



Anchor boxes at each predefined location in each feature map



객체탐지 (Object Detection)

YOLO 모델의 구조

